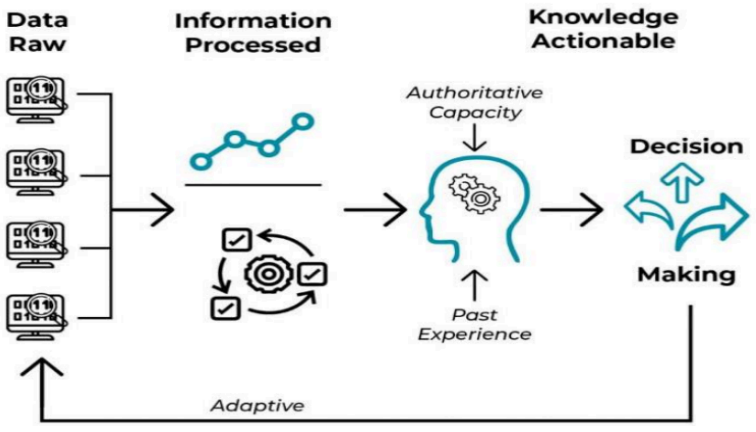
UNIT-4 KNOWLEDGE REPRESENTATION

1. Procedural Versus Declarative knowledge
2. Using Predicate logic
3. representing facts in logic
4. functions and predicates
5. Conversion to clause form
6. Resolution in propositional logic
7. Resolution in predicate logic
8. Unification

Knowledge Representation (KR) in Artificial Intelligence

1. **Purpose of KR:**
   - Enables machines to understand, reason, and interpret knowledge similar to humans.
   - Focuses on how machines can perform actions using knowledge and reasoning.
2. **Key Concepts:**
   - KR is not just about storing data but also about enabling machines to learn and behave intelligently.
   - It is central to the thinking process of AI agents, helping them exhibit intelligent behavior.
3. **Types of Knowledge in AI Systems:**
   - **Objects:** Information about objects, e.g., cars have wheels, pianos have keys.
   - **Events:** Knowledge of past events like wars, famines, societal advancements.
   - **Performance:** Understanding how to perform tasks or actions.
   - **Meta-knowledge:** Knowledge about what we know.
   - **Facts:** Truths about the real world, represented logically.
   - **Knowledge-Base:** A collection of sentences (in a technical sense) that represent knowledge.



Types of Knowledge:

Declarative Knowledge is knowledge *about* something. It is information that can be stated in words or symbols and is primarily concerned with *what* things are.

Characteristics:

- Includes **facts, concepts, and objects**.
- Often expressed in **declarative sentences** (e.g., "A car has four wheels" or "The Earth revolves around the Sun").
- It is **static** knowledge, meaning it describes things but doesn't specify how to act on that information.
- Easier to understand and communicate, making it simpler than procedural knowledge.

Examples:

- Knowing that Paris is the capital of France.
- Understanding that water boils at 100°C.
- Recognizing that a cat is a mammal.

Procedural Knowledge refers to knowing *how to do* something. It is the knowledge of processes, methods, or steps required to perform specific tasks or actions.

Characteristics:

- Also known as **imperative knowledge**, as it instructs what to do.
- It is **dynamic** because it involves action, like executing a task or following a set of rules.
- Often harder to articulate because it's focused on doing rather than describing.
- Includes **rules, strategies, procedures, and algorithms** that can be directly applied to tasks.

Examples:

1. Knowing how to ride a bicycle.
2. Understanding the steps to solve a math problem.
3. Executing a recipe to bake a cake.

Feature	Declarative Knowledge	Procedural Knowledge
Focus	Knowing *what*	Knowing *how*
Nature	Static (descriptive)	Dynamic (action-oriented)
Examples	Facts, concepts (e.g., "Paris is in France")	Steps, processes (e.g., "How to ride a bike")
Application	Cannot be directly applied to tasks	Can be directly applied to tasks
Ease of Expression	Easier to express and communicate	Harder to articulate (often implicit)

Meta-knowledge:

- Knowledge *about* other types of knowledge.
- Helps organize and understand how different types of knowledge interact.

Heuristic Knowledge:

- Knowledge based on **rules of thumb** or **expert experience**.
- Derived from past experiences, effective but not always guaranteed.
- Used for practical problem-solving when exact solutions are unavailable.

Structural Knowledge:

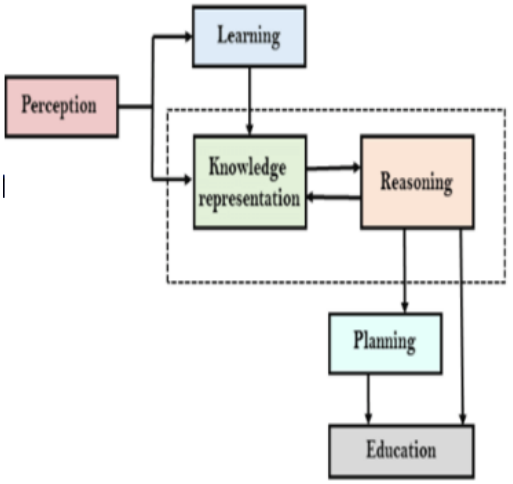
- Basic knowledge for **problem-solving**.
- Describes relationships between concepts (e.g., "part of," "kind of").
- Explains how different objects or concepts are connected.

The Relation Between Knowledge and Intelligence

- **Role of Knowledge in Intelligence:**
  - Knowledge about the real world is crucial for intelligence.
  - It enables intelligent behavior in AI agents by providing context and understanding.
- **Importance for AI Agents:**
  - An AI agent can act intelligently only if it has **knowledge or experience** about the input it receives.
  - This knowledge allows the agent to make informed decisions and responses.

AI Knowledge Cycle

1. **Perception:** Collecting data from the environment.
2. **Learning:** Improving decisions based on data and experiences.
3. **Knowledge Representation and Reasoning:** Organizing knowledge and making decisions.
4. **Planning:** Developing strategies to achieve goals.
5. **Execution:** Carrying out actions to interact with the environment.

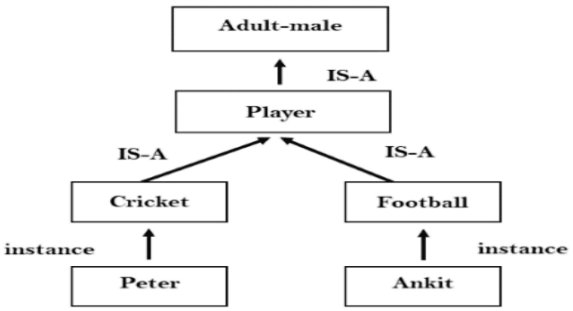


Operations Performed by Knowledge-Based Agent (KBA)

1. **TELL:**Informs the knowledge base about the agent's perceptions from the environment.
2. **ASK:**Queries the knowledge base to decide what action should be performed next.
3. **Perform:**Executes the selected action based on the knowledge base's response.

Approaches to Knowledge Representation

1. **Simple Relational Knowledge:**
    - The simplest method of storing facts using relationships between entities.
    - Example: Representing the relationship between different entities in a database.
2. **Inheritable Knowledge:**
    - Data is stored in a hierarchy of classes, allowing inheritance of properties or characteristics.
    - Example: A class hierarchy where a subclass inherits attributes from a parent class.

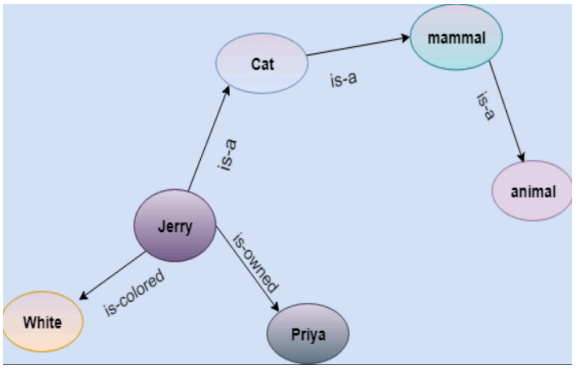


3. **Inferential Knowledge:**
    - Represents knowledge using formal logic.
    - Used to derive new facts from existing ones.
    - Example:
        - Statements:
            - Marcus is a man
            - All men are mortal
        - Representation:
            - man(Marcus)
            - $\forall x \, man(x) \rightarrow mortal(x)$
4. **Procedural Knowledge:**
    - Uses small programs or codes that describe how to perform specific tasks.
    - Typically represented using **If-Then** rules for decision-making.

Techniques of Knowledge Representation

1. **Logical Representation:**
    - A formal language with clear, unambiguous rules used to represent propositions.
    - Conclusions are drawn based on conditions.
        - **Propositional Logic**: Deals with simple statements or propositions.
        - **Predicate Logic**: Uses predicates and quantifiers for more complex statements.

2. **Semantic Network Representation:**
    - An alternative to predicate logic, representing knowledge as a **graphical network**.
    - **Nodes** represent objects, and **arcs** represent relationships between objects.
    - Example:
        - "Jerry is a cat."
        - "Jerry is a mammal."
        - "Jerry is owned by Priya."
        - "Jerry is brown colored."
        - "All mammals are animals."



3. **Frame Representation:**
    - A frame is a structured record that contains **attributes** and their corresponding **values** to describe an entity.
    - Example:
        - **Frame for a Book**
            - **Slots**: Title, Genre
            - **Filters**: "Artificial Intelligence", "Computer"

Slots	Filters
Title	Artificial Intelligence
Genre	Computer Science
Author	Peter Norvig
Edition	Third Edition
Year	1996
Page	1152

4. **Production Rules:**
    - Consist of **condition-action** pairs in the form "If condition, then action."
    - Example Rules:
        - **IF** (at bus stop AND bus arrives) **THEN** (get into the bus).
        - **IF** (on the bus AND paid AND empty seat) **THEN** (sit down).
        - **IF** (on bus AND unpaid) **THEN** (pay charges).
        - **IF** (bus arrives at destination) **THEN** (get down from the bus).

Propositional Logic (PL)

1. **Definition:**
    - Propositional Logic is the simplest form of logic where statements are made using **propositions**.
    - A **proposition** is a declarative statement that is either true or false.
2. **Properties of Propositional Logic:**
    - Also called **Boolean Logic**, as it works with **0 (false)** and **1 (true)**.
    - A formula that is always **true** is called a **tautology**.
    - A formula that is always **false** is called a **contradiction**.
    - **Non-propositions** include questions, commands, or opinions, like "Where is Rohini?" or "What is your name?" These are not propositions because they do not have a definite true or false value.

Syntax of Propositional Logic: The syntax defines the rules for constructing valid sentences in propositional logic. There are two main types of propositions:

1. **Atomic Propositions:**
    - **Simple** propositions consisting of a single symbol.
    - They must be either **true** or **false**.
    - Example:
        - "2+2 is 4" is true → Atomic proposition.
        - "The Sun is cold" is false → Atomic proposition.
2. **Compound Propositions:**
    - Constructed by combining atomic propositions using logical connectives (e.g., AND, OR, NOT).
    - Parentheses are used to clarify structure.
    - Example:
        - "It is raining today, and the street is wet."
        - "Ankit is a doctor, and his clinic is in Mumbai."

Logical Connectives in Propositional Logic

1. **Negation (¬):**
    - Negation is used to **reverse the truth value** of a proposition.
    - A sentence like **¬P** means "not P", where P is a proposition.
    - **Literals** can be either **positive** (P) or **negative** (¬P).
2. **Conjunction (∧):**
    - A conjunction is a sentence with the **AND** connective (∧).
    - Represents that both propositions must be true for the entire sentence to be true.
    - Example:
        - "Rohan is intelligent and hardworking."
        - Let P = "Rohan is intelligent" and Q = "Rohan is hardworking".
        - The conjunction is represented as **P ∧ Q**.

3. **Disjunction (∨):**
    - A disjunction is a sentence with the **OR** connective (∨).
    - Represents that at least one of the propositions must be true for the entire sentence to be true.
    - Example:
        - "Ritika is a doctor or engineer."
        - Let P = "Ritika is a doctor" and Q = "Ritika is an engineer".
        - The disjunction is represented as **P ∨ Q**.

Truth Table				
P	Q	Conjunction AND P ∧ Q	Disjunction OR P ∨ Q	Negation NOT P ~P
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

4. **Implication (→):**
    - An implication is a sentence of the form **P → Q** (if P then Q).
    - It suggests that if the first proposition is true (P), then the second proposition (Q) must also be true.
    - Example:
        - "If it is raining, then the street is wet."
        - Let P = "It is raining" and Q = "The street is wet".
        - The implication is represented as **P → Q**.

P	Q	P → Q
T	T	T
T	F	F
F	T	T
F	F	T

5. **Biconditional (⇔):**
    - A biconditional sentence is of the form **P ⇔ Q** (P if and only if Q).
    - It means both P and Q must either both be true or both be false.
    - Example:
        - "If I am breathing, then I am alive."
        - Let P = "I am breathing" and Q = "I am alive".
        - The biconditional is represented as **P ⇔ Q**.

P	Q	P ⇔ Q
T	T	T
T	F	F
F	T	F
F	F	T

Following is the summarized table for Propositional Logic Connectives:

Connective symbols	Word	Technical term	Example
∧	AND	Conjunction	A ∧ B
∨	OR	Disjunction	A ∨ B
→	Implies	Implication	A → B
⇔	If and only if	Biconditional	A ⇔ B
¬ or ~	Not	Negation	¬ A or ¬ B

Properties of Operators:

1. Commutativity:
   - AND (∧): $P \wedge Q = Q \wedge P$
   - OR (∨): $P \vee Q = Q \vee P$
2. Associativity:
   - AND (∧): $(P \wedge Q) \wedge R = P \wedge (Q \wedge R)$
   - OR (∨): $(P \vee Q) \vee R = P \vee (Q \vee R)$
3. Identity Element:
   - AND (∧): $P \wedge \text{True} = P$
   - OR (∨): $P \vee \text{True} = \text{True}$
4. Distributive:
   - $P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$
   - $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$
5. De Morgan's Law:
   - $\neg(P \wedge Q) = \neg P \vee \neg Q$
   - $\neg(P \vee Q) = \neg P \wedge \neg Q$
6. Double-Negation Elimination:
   - $\neg(\neg P) = P$

A **WFF (Well-formed Formula)** is a logically valid expression in propositional logic, formed according to specific syntax rules. It consists of:

- **Atomic propositions** (e.g., P, Q)
- **Logical connectives** (e.g., AND ∧, OR ∨, NOT ¬, IMPLIES →)
- **Parentheses** for proper structure.

Examples of WFFs:

- **P**
- **P ∧ Q**
- **(P ∨ Q) → R**
- **¬(P ∧ Q)**

Invalid WFF:

- **P ∧ ∨ Q**
- **P →**

Bracket Simplification involves reducing logical expressions by removing unnecessary parentheses and applying logical rules:

1. **Eliminate Redundant Parentheses**: Remove unnecessary brackets.
   - Example: $(P \land Q) \lor R$ becomes $P \land Q \lor R$.
2. **Apply Commutativity**: Swap operands.
   - Example: $P \land Q$ is the same as $Q \land P$.
3. **Apply Associativity**: Rearrange terms.
   - Example: $(P \land Q) \land R$ becomes $P \land (Q \land R)$.
4. **Apply Distributivity**: Distribute one operator over another.
   - Example: $P \land (Q \lor R)$ becomes $(P \land Q) \lor (P \land R)$.
5. **Use De Morgan's Laws**: Simplify negations.
   - Example: $\neg(P \land Q)$ becomes $\neg P \lor \neg Q$.
6. **Simplify Double Negations**: $\neg(\neg P)$ becomes P.

Limitations of Propositional Logic:

- Propositional logic cannot represent relations like **ALL**, **SOME**, or **NONE**.
    - Example:
        - "All the girls are intelligent" – cannot be expressed directly in propositional logic.
        - "Some apples are sweet" – propositional logic does not support partial relationships like "some."
- **Expressive Power**: Propositional logic has limited expressive power because it cannot handle quantifiers or complex relationships.

Inference is the process of drawing conclusions from evidence or facts.

Terminologies Related to Inference Rules:

1. **Implication**:
    - Represented as P→Q, meaning **"If P, then Q."**
    - It is a **Boolean expression**.
2. **Converse**:
    - The converse of an implication reverses the order of the propositions, written as Q→P
3. **Contrapositive**:
    - The contrapositive is the negation of the converse, written as ¬Q→¬P
    - It means **"If not Q, then not P."**
4. **Inverse**:
    - The inverse is the negation of the implication, written as ¬P→¬Q
    - It means **"If not P, then not Q.**

Types of Inference Rules:

1. **Modus Ponens (Affirming the Antecedent)**:
   - **Rule**: If P and $P \rightarrow Q$ are true, then Q is true.
   - **Example**:
     - Statement-1: "If I am sleepy, then I go to bed" ($P \rightarrow Q$)
     - Statement-2: "I am sleepy" (P)
     - Conclusion: "I go to bed" (Q)
2. **Modus Tollens (Denying the Consequent)**:
   - **Rule**: If $P \rightarrow Q$ is true and $\neg Q$ is true, then $\neg P$ is true.
   - **Example**:
     - Statement-1: "If I am sleepy, then I go to bed" ($P \rightarrow Q$)
     - Statement-2: "I do not go to bed" ($\neg Q$)
     - Conclusion: "I am not sleepy" ($\neg P$)
3. **Hypothetical Syllogism**:
   - **Rule**: If $P \rightarrow Q$ and $Q \rightarrow R$ are true, then $P \rightarrow R$ is true.
   - **Example**:
     - Statement-1: "If you have my home key, then you can unlock my home" ($P \rightarrow Q$)
     - Statement-2: "If you can unlock my home, then you can take my money" ($Q \rightarrow R$)
     - Conclusion: "If you have my home key, then you can take my money" ($P \rightarrow R$)
4. **Disjunctive Syllogism**:
   - **Rule**: If $P \vee Q$ is true and $\neg P$ is true, then Q is true.
   - **Example**:
     - Statement-1: "Today is Sunday or Monday" ($P \vee Q$)
     - Statement-2: "Today is not Sunday" ($\neg P$)
     - Conclusion: "Today is Monday" (Q)

5. **Addition**:
    - **Rule**: If P is true, then $P \vee Q$ is true.
    - **Example**:
        - Statement: "I have a vanilla ice cream" (P)
        - Conclusion: "I have vanilla or chocolate ice cream" ($P \vee Q$)
6. **Simplification**:
    - **Rule**: If $P \wedge Q$ is true, then P or Q is true.
    - **Example**:
        - Statement: "I have a car and a bike" ($P \wedge Q$)
        - Conclusion: "I have a car" (P) or "I have a bike" (Q)
7. **Resolution**:
    - **Rule**: If $P \vee Q$ and $\neg P \vee R$ are true, then $Q \vee R$ is true.
    - **Example**:
        - Statement-1: "It is sunny or cloudy" ($P \vee Q$)
        - Statement-2: "It is not sunny or it is raining" ($\neg P \vee R$)
        - Conclusion: "It is cloudy or it is raining" ($Q \vee R$)

First-Order Logic (FOL)/Predicate Logic :

Limitations of Propositional Logic (PL):Propositional logic can only represent facts that are either true or false, but it cannot handle complex sentences or relationships, like "Some humans are intelligent" or "Sachin likes cricket."

First-Order Logic (FOL):

1. Extension of PL: FOL (also known as Predicate Logic) extends propositional logic to represent more complex statements.
2. Key Elements:
    - Objects: Specific entities (e.g., people, colors, numbers).
    - Relations: Unary relations (e.g., red, round) or n-ary relations (e.g., sister of, father of).
    - Functions: Mapping entities to other entities (e.g., father of, best friend of).

Two Main Parts of FOL:

Syntax: Rules for forming valid expressions.

Semantics: Meaning and interpretation of those expressions.

Basic Syntactic Elements in FOL:

- **Constants**: Specific objects or entities (e.g., 1, 2, A, John, Mumbai, cat).
- **Variables**: Represent unspecified entities (e.g., x, y, z).
- **Predicates**: Represent relations (e.g., Brother, Father, >).
- **Functions**: Represent mappings (e.g., sqrt, LeftLegOf).
- **Connectives**: Logical operators (e.g., $\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$).
- **Equality**: Denoted as ==.
- **Quantifiers**: Express the scope of a variable (e.g., $\forall$, $\exists$).

Atomic Sentences:

- Formed from a predicate followed by terms.
- **Example**:
    - **Ravi and Ajay are brothers**: Brothers(Ravi, Ajay).
    - **Chinky is a cat**: Cat(Chinky).

Complex Sentences:

- Created by combining atomic sentences using logical connectives.
- Can be broken into:
    - **Subject**: The main entity (e.g., x in "x is an integer").
    - **Predicate**: A relation binding the entities (e.g., "is an integer").

Quantifiers in First-Order Logic:

- **Universal Quantifier (∀)**: Specifies that a statement is true for every instance of something.
    - **Example**: "All men drink coffee" is represented as ∀x (Man(x) → Drink(x, coffee)).
- **Existential Quantifier (∃)**: Indicates that a statement is true for at least one instance of something.
    - **Example**: "Some boys are intelligent" is represented as ∃x (Boy(x) ∧ Intelligent(x)).

Properties of Quantifiers:

- **Universal Quantifiers**: ∀x∀y is equivalent to ∀y∀x.
- **Existential Quantifiers**: ∃x∃y is equivalent to ∃y∃x.
- **Nested Quantifiers**: Can combine both quantifiers in statements.
    - **Example**: ∃xy ∀x ∀y ((x < 0) ∧ (y < 0) → (xy = 8)).

Key Points:

- **Universal Quantifier (∀)**: Expresses "for all" or "every."
- **Existential Quantifier (∃)**: Expresses "there exists" or "at least one."

First-order logic is powerful because it allows expressing complex relationships and constraints that propositional logic cannot, and it uses syntax and semantics similar to natural language.

Here are the sentences translated into First-Order Logic (FOL):

1. **Coconut-crunchy is a biscuit.**
    - Biscuit(coconut-crunchy)
2. **Mary is a child who takes coconut-crunchy.**
    - Child(Mary) ∧ Takes(Mary, coconut-crunchy)
3. **John loves children who take biscuits.**
    - ∀X ((Child(X) ∧ ∃Y (Takes(X, Y) ∧ Biscuit(Y))) → Loves(John, X))
4. **John loves Mary.**
    - Loves(John, Mary)

Explanation of each sentence in FOL:

1. The first sentence states that "Coconut-crunchy is a biscuit," so we directly use the predicate Biscuit(coconut-crunchy) to represent this fact.

2. The second sentence is about Mary being a child who takes coconut-crunchy, so we combine Child(Mary) and Takes(Mary, coconut-crunchy) using the logical conjunction $\wedge$.
3. The third sentence asserts that John loves children who take biscuits. It uses a universal quantifier for children ($\forall X$) and an existential quantifier for the items (biscuits) they take ($\exists Y$), with a conditional relation ($\rightarrow$) expressing that if X is a child and takes a biscuit, then John loves X.
4. The fourth sentence simply states that John loves Mary, represented as Loves(John, Mary).

In **Predicate Logic** (also known as **First-Order Logic**), the operators (also called logical connectives) are similar to those in **Propositional Logic**, but they are applied to predicates, functions, and quantifiers. Here's a list of the key operators in **Predicate Logic**:

1. Logical Connectives (used to form compound logical statements)

These connectives combine simple logical statements (atomic formulas) into more complex ones.

- **Negation (¬)**: Denoted by ¬, it represents "not". If P is a predicate, ¬P means "not P".
    - Example: ¬Likes(John, Mary) means "John does not like Mary."
- **Conjunction ($\wedge$)**: Denoted by $\wedge$, it represents "and". It connects two predicates or logical statements, and it is true if both components are true.
    - Example: Likes(John, Mary) $\wedge$ Loves(John, Mary) means "John likes Mary and John loves Mary."
- **Disjunction ($\vee$)**: Denoted by $\vee$, it represents "or". It is true if at least one of the components is true.
    - Example: Likes(John, Mary) $\vee$ Loves(John, Mary) means "John likes Mary or John loves Mary (or both)."
- **Implication ($\rightarrow$)**: Denoted by $\rightarrow$, it represents "if... then". It means if the first statement is true, then the second one must be true.
    - Example: Likes(John, Mary) $\rightarrow$ Loves(John, Mary) means "If John likes Mary, then John loves Mary."
- **Biconditional ($\Leftrightarrow$)**: Denoted by $\Leftrightarrow$, it represents "if and only if". It means both components must either be true or false.
    - Example: Likes(John, Mary) $\Leftrightarrow$ Loves(John, Mary) means "John likes Mary if and only if John loves Mary."

2. Quantifiers (used to express the scope of variables)

Universal Quantifier (∀): Denoted by ∀, it represents "for all" or "for every". It specifies that the statement it precedes is true for all possible values of the variable.

- Example: ∀x (Human(x) → Mortal(x)) means "All humans are mortal."

Existential Quantifier (∃): Denoted by ∃, it represents "there exists" or "there is at least one". It specifies that there is at least one value of the variable that makes the statement true.

- Example: ∃x (Human(x) ∧ Loves(x, Mary)) means "There exists a human who loves Mary."

3. Equality (=):

The equality symbol = is used to express that two terms refer to the same object or entity.

Example: x = y means "x and y are the same."

4. Functions:

In First-Order Logic, functions return specific values for a given set of inputs (terms).

Example: FatherOf(John) could represent the father of John.

5. Predicates:

Predicates are relations between objects or between an object and a property. They are used to express facts or properties of the world.

Example: Likes(John, Mary) expresses the fact that "John likes Mary."

Operator	Symbol	Meaning	Example
Negation	¬	Not	`¬Likes(John, Mary)` (John does not like Mary)
Conjunction	∧	And	`Likes(John, Mary) ∧ Loves(John, Mary)`
Disjunction	∨	Or	`Likes(John, Mary) ∨ Loves(John, Mary)`
Implication	→	If...then	`Likes(John, Mary) → Loves(John, Mary)`
Biconditional	⇔	If and only if	`Likes(John, Mary) ⇔ Loves(John, Mary)`
Universal Quantifier	∀	For all	`∀x Human(x) → Mortal(x)` (All humans are mortal)
Existential Quantifier	∃	There exists	`∃x Human(x) ∧ Loves(x, Mary)` (Some human loves Mary)
Equality	=	Equality	`x = y` (x is equal to y)

Examples of FOL using quantifier:

1. All birds fly.

- **∀x bird(x) →fly(x).**

2. Every man respects his parent.

∀x man(x) → respects (x, parent).

3. Some boys play cricket.

∃x boys(x) → play(x, cricket).

4. Not all students like both Mathematics and Science.

¬∀ (x) [student(x) → like(x, Mathematics) ∧ like(x, Science)].

5. Only one student failed in Mathematics.

∃(x) [student(x) → failed (x, Mathematics) ∧∀ (y) [¬(x==y) ∧ student(y) → ¬failed (Y, Mathematics)].

Clausal form (CNF) is a way of expressing logical sentences in a form that is easier to work with in automated reasoning systems. The formula is a conjunction of clauses, where each clause is a disjunction (OR) of literals. Below is the process for converting a sentence into CNF:

Steps for Converting a Sentence into Clauses (CNF):

Step I: Elimination of the → operator

- The implication operator (→) is replaced with the logical equivalence:
  - $P \rightarrow Q$ is rewritten as $\neg P \vee Q$.

  Example:

  - $P \rightarrow Q$ becomes $\neg P \vee Q$.

Step II: Reduction of the scope of negation

- Move negations inward using De Morgan's Laws and double negation.
  - $\neg(P \vee Q)$ becomes $\neg P \wedge \neg Q$ (De Morgan's Law).
  - $\neg(\neg P)$ becomes P (double negation).

  Example:

  - $\neg\neg P$ becomes P.
  - $\neg(P \vee Q)$ becomes $\neg P \wedge \neg Q$.

Step III: Renaming of variables within the scope

- Rename variables to avoid clashes, especially when variables appear multiple times with different scopes.

  Example:

  - If $\forall x\,(P(x) \rightarrow Q(x))$ and $\forall y\,(P(y) \rightarrow R(y))$, we might rename one variable: $\forall x\,(P(x) \rightarrow Q(x))$ and $\forall z\,(P(z) \rightarrow R(z))$.

Step IV: Moving quantifiers to the front

- Bring quantifiers (∀ and ∃) to the front of the expression. Universal quantifiers (∀) should appear first.

  Example:

  - $\forall x\,\exists y\,(P(x, y))$ becomes $\forall x\,\exists y\,(P(x, y))$, but with the quantifiers placed clearly at the front of the expression.

Step V: Replacing existential quantifier as Skolem function

- Replace existential quantifiers with Skolem functions. This step ensures that the expression is in Skolem normal form.

  Example:

  - $\exists y\,(P(x, y))$ becomes $P(x, f(x))$, where $f(x)$ is a Skolem function that replaces y.

Step VI: Putting the resulting expression in Conjunctive Normal Form (CNF)

- The expression must be in CNF, meaning it is a conjunction of clauses, where each clause is a disjunction of literals. This step ensures the expression is structured in this way.

  Example:

  - Convert $P \rightarrow (Q \vee R)$ to CNF:
    - $\neg P \vee (Q \vee R)$, which is in CNF form.

Step VII: Writing one clause per line

- Write each clause as a line in the CNF formula. A clause is a disjunction of literals (e.g., $P \vee Q$).

  Example:

  - For the formula $(P \vee Q) \wedge (R \vee S)$, write:
    - Clause 1: $P \vee Q$
    - Clause 2: $R \vee S$

Example:

Let's apply this process to a simple sentence:

Sentence: $\forall x \, (P(x) \rightarrow \exists y \, Q(x, y))$

1. **Step I (Eliminate the → operator):**
   - $\forall x \, (\neg P(x) \vee \exists y \, Q(x, y))$
2. **Step II (Reduce negation scope):**
   - No further changes needed for negations.
3. **Step III (Rename variables within scope):**
   - Rename y to avoid clash if required.
4. **Step IV (Move quantifiers to the front):**
   - Quantifiers are already at the front: $\forall x \, \exists y$.
5. **Step V (Replace existential quantifier with Skolem function):**
   - Replace $\exists y \, Q(x, y)$ with $Q(x, f(x))$, where $f(x)$ is a Skolem function.
   - The sentence becomes: $\forall x \, (\neg P(x) \vee Q(x, f(x)))$.
6. **Step VI (CNF):**
   - The expression is already in CNF form: $\neg P(x) \vee Q(x, f(x))$.
7. **Step VII (Write one clause per line):**
   - Clause 1: $\neg P(x) \vee Q(x, f(x))$.

This completes the conversion to CNF.